Field Hours

CSCE 628 Computational Biology

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Strings, alphabets, and frames

You treat a genome as a string over a small alphabet. DNA uses 4 letters, A, C, G, and T, and the complement swaps A with T and C with G while reversing direction. A read stored from the opposite strand will not match the reference until you build that reverse complement. Proteins are a different string: 20 amino acids, written after the cell has translated codons of length 3.

A codon is 3 bases and codes for one amino acid or a stop. On one strand you can start at position 0, 1, or 2, which gives 3 forward frames. The reverse complement has 3 more, so a double-stranded stretch has 6 frames you may have to translate. A shift of 1 base changes every codon downstream. That is why you do not compare protein strings until you have chosen the frame the bases were read in.

One DNA stretch, six translations.

DNA string of 4 letters

Reverse complement of that string

3 starts on the given strand

3 starts on the opposite strand

Reverse complement of ACGT

1. Write the string ACGT. Replace each letter: A becomes T, C becomes G, G becomes C, T becomes A. You now have TGCA.

2. Reverse that result. The reverse complement is ACGT. This particular string is its own reverse complement.

3. A one-base shift still matters after the complement. Reading ATGAAA from index 1 is not the same codon sequence as reading it from index 0.

Problem 1. What is the reverse complement of ATG, and how many forward frames does a single strand give you?

Answer. The complement of ATG is TAC, and the reverse of TAC is CAT. A single strand gives you 3 forward frames, starting at positions 0, 1, and 2.

Problem 2. Why can a shift of 1 base make a protein comparison meaningless even when the letters are unchanged?

Answer. Codons are read in groups of 3. Starting one base later regroups every codon downstream, so the translated amino acids are a different string. You have to choose the frame before you compare

proteins.

Lecture 2 · 90 minutes

Edit distance

You measure how far two strings are by the cheapest script of edits. Insertion, deletion, and substitution each have a cost you set. With every cost equal to 1, distance is the size of the cheapest script. CAT and CAR differ by one substitution at the end, so the distance is 1. CAT and AT differ by deleting C, so that distance is 1 as well.

An alignment writes the edits in columns. Two letters in a column are a match or a substitution, and a dash is a gap. A gap is an insertion against one string and a deletion against the other. For strings of length 3 and 3, you need at least 3 columns and at most 6, and you never place 2 dashes in the same column. You want the cheapest alignment, not the first one you sketch.

Two edits, both cost 1.

CAT against CAR

Last letters differ

One substitution

Distance 1

CAT against AT

Drop the leading C

One deletion

Distance 1

Align CAT with CAR at unit cost

1. Match cost is 0, and substitution, insertion, and deletion each cost 1.

2. Pair C with C and A with A. Those two columns cost 0.

3. Pair T with R. That substitution costs 1, so the alignment costs 1, which is the distance.

Problem 1. With every edit cost equal to 1, what is the distance between AC and ABC?

Answer. 1. Insert B between A and C. The other letters match, so one insertion is enough, and you cannot do it in 0 edits because the strings differ.

Problem 2. Two strings each have length 3. Why is an alignment of 7 columns impossible under the rule you are using?

Answer. Each letter appears in some column, so you need at least 3 columns, and you never stack 2 dashes in one column, so you cannot exceed 6. A 7th column would require two dashes or a repeated letter.

Lecture 3 · 90 minutes

Global and local alignment

You fill a matrix whose cell (i, j) is the best score of the first i letters against the first j letters. Needleman-Wunsch aligns the strings globally. Each cell takes the best of three moves: a gap in the first string, a gap in the second, or a pairing of the two current letters. With match cost 0, substitution cost 1, and gap cost 1, the bottom-right cell is the edit distance.

Smith-Waterman changes one rule: a cell that would go negative is reset to 0, and you may start a new local alignment there. You report the best cell anywhere in the matrix, not the corner. That is the right mechanism when a gene of length 40 sits inside a chromosome of length 4000 and the flanking bases should not pay gap costs. A global alignment would force those flanks into the score.

One cell, three incoming moves.

1. Gap in the first string
2. Gap in the second string
3. Pair the two letters
4. Keep the cheapest move

Corner cell for CAT and CAR

1. Costs are match 0, substitution 1, gap 1. The cell for the first C against the first C stores 0.
2. The cell for prefix CA against prefix CA also stores 0, because both pairings match.
3. The final T against R is a substitution, so the bottom-right cell stores 1. That 1 is the global distance.

Problem 1. Under match 0, substitution 1, and gap 1, the bottom-right cell of CAT against CAR is 1. What does that cell mean?

Answer. It is the edit distance of the whole strings. Needleman-Wunsch reports the corner, and the cheapest global alignment substitutes T for R and matches the other two letters.

Problem 2. A gene is 40 bases inside a chromosome of 4000. Why do you reset negative cells to 0 instead of forcing a global alignment?

Answer. A global alignment charges the flanking bases, which can bury a good local match. Smith-Waterman resets a negative cell to 0 so a local alignment can start there, and you read the best cell in the matrix rather than the corner.

Lecture 4 · 90 minutes

Substitution scores for proteins

DNA match-or-mismatch is a blunt score once you have translated to protein. You look up a pair of amino acids in a substitution matrix instead of charging every change the same cost. The matrix entry is high when that substitution is common in real proteins and low when it is rare. In a toy matrix, A against A scores 2, and A against B scores -1. You add the column scores to score the alignment.

Gaps still have their own penalty, a larger cost to open a gap than to extend it by 1 more letter. You choose the matrix for the question you have. A matrix built from close homologs punishes substitutions harder than a matrix built from distant ones. If two candidate alignments of the same pair score 8 and 5 under your matrix, you keep 8. The number is not a probability until you say how random strings of that length score.

Same change, two alphabets.

DNA cost

Match costs 0

Any change costs 1

Letters are only 4

Protein matrix

A with A scores 2

A with B scores -1

Letters are 20

Score AA against AB

1. Use the toy matrix: identical letters score 2, and A against B scores -1. Open no gaps.

2. Column 1 pairs A with A and scores 2. Column 2 pairs A with B and scores -1.

3. The alignment scores 2 + (-1) = 1. A second alignment that scores 5 would outrank this one.

Problem 1. Under the toy matrix, identical letters score 2 and A against B scores -1. What is the score of AAA against ABA with no gaps?

Answer. 3. The columns score 2, -1, and 2. The sum is 2 - 1 + 2 = 3.

Problem 2. Two alignments of the same protein pair score 8 and 5. Which do you keep, and why is 8 not yet a probability?

Answer. You keep 8, the higher matrix score. It becomes a probability only after you say how random strings of that length score. The raw sum is not that probability.

Lecture 5 · 90 minutes

Seeds before a full alignment

Full dynamic programming against every database sequence is too slow for a query you repeat. BLAST first requires an exact seed shared by the query and a subject. Protein search often uses a seed of length 3. DNA search often uses a seed of length 11, because 11 letters are already specific in an alphabet of 4. A subject with no seed is not passed to the slower alignment step.

Around each seed you extend both ways and stop when the score falls more than a dropoff below its best so far. A dropoff of 5 means a dip of 6 ends that extension. You keep the hit only if the finished score clears a threshold you set, such as 20. A seed of length 3 can still die in a bad neighborhood. You can miss a true homolog that

shares no exact word of length 3.

A hit has to survive two gates.

Find an exact seed -> Extend while the score holds -> Drop a dip past 5 -> Keep scores of at least 20

A protein seed of length 3

1. The query contains the word MKM. A database protein contains MKM starting at one position. That shared word is a seed of length 3.

2. You extend left and right. The running score peaks, then falls by 6, which is more than the dropoff of 5, so extension stops.

3. The finished score is 18. The threshold is 20, so you discard the hit even though the seed matched.

Problem 1. Why is a DNA seed often length 11 while a protein seed is often length 3?

Answer. DNA has an alphabet of 4, so a word of length 11 is already specific. Protein has an alphabet of 20, so a word of length 3 is specific enough to use as the seed. Subjects with no seed skip the slow alignment.

Problem 2. A seed matches and the extension score peaks, then falls by 6 from that peak, and finishes at 18. Your dropoff is 5 and your threshold is 20. Do you keep the hit?

Answer. No. A dip of 6 is past the dropoff of 5, so extension stops, and 18 is below the threshold of 20. An exact seed is not by itself a reported hit.

Lecture 6 · 90 minutes

Reads, k-mers, and a contig

A sequencer gives you short reads, not a finished chromosome. Shotgun assembly overlaps those reads until the overlaps spell a longer sequence. In a de Bruijn graph you break each read into k-mers, words of length k. Consecutive k-mers overlap by k minus 1 letters and form a path you can follow. With k equal to 3, the read ATGCT yields the 3 k-mers ATG, TGC, and GCT, and that path spells ATGCT again.

A second read, GCTA, with the same k of 3, yields GCT and CTA. GCT is already a node, so the reads merge into one path that spells ATGCTA, length 6. You needed a shared k-mer. If k is too small, a repeat of length 2 glues unrelated regions together. If k is larger than the overlap you truly have, the shared node never appears and the reads stay in 2 components.

Two reads become one path.

1. Break each read into 3-mers

2. Share the node GCT

3. Spell ATGCTA on the path

Assemble ATGCT with GCTA

1. k is 3. ATGCT gives ATG, TGC, and GCT. Consecutive k-mers overlap by 2 letters.

2. GCTA gives GCT and CTA. GCT is already present, so the second read joins the path.

3. The merged path spells ATGCTA, which has length 6. Without the shared 3-mer the reads would stay separate.

Problem 1. With k = 3, which 3-mers do you take from ATGCT, and what string does that path spell?

Answer. ATG, TGC, and GCT. Consecutive 3-mers overlap by 2 letters, and the path spells ATGCT.

Problem 2. You add the read GCTA. What contig do you spell at k = 3, and what happens at k = 4?

Answer. At k = 3 the reads share GCT and spell ATGCTA, length 6. At k = 4, ATGCT yields ATGC and TGCT, while GCTA yields only GCTA. No node is shared, so the reads stay in 2 components.

Lecture 7 · 90 minutes

From a frame to a peptide

You translate DNA only after you pick a frame. Take ATGAAA, which has length 6, so it holds 2 codons if you start at position 0: ATG and AAA. ATG is a common start, and AAA codes for lysine in the standard table, so this frame yields a dipeptide you can write as MK if you use the one-letter code. Starting at position 1 uses different groups and is a different product.

An open reading frame runs from a start codon to an in-frame stop, and its base length is divisible by 3. You scan 6 frames because the given strand may be backwards. An ORF of 99 codons is 297 bases. For proteomics you digest the predicted protein in the model: trypsin cuts after K and after R. A cut site inside the chain means the mass you look for is the peptide mass, not the mass of the whole protein.

Bases to a mass you can search.

Pick 1 of 6 frames -> Translate codons of length 3 -> Cut after K and after R -> Search the peptide mass

Translate ATGAAA in frame 0

1. Length 6 divides by 3, so frame 0 has 2 codons: ATG and AAA.

2. ATG becomes M and AAA becomes K. The peptide is MK.

3. Frame 1 does not group the bases that way. You do not report MK for frame 1.

Problem 1. An open reading frame has 99 codons. How many coding bases is that, and why must the count be divisible by 3?

Answer. 297 bases, because 99 times 3 is 297. Codons are length 3, so a base count that is not divisible by 3 is not an integer number of codons in that frame.

Problem 2. Trypsin cuts after K and after R. Why do you search a peptide mass rather than the mass of the whole translated protein?

Answer. A cut site inside the chain breaks the protein into peptides. The spectrum matches a peptide mass, not the mass of the intact chain, whenever a K or an R falls inside it.

Lecture 8 · 90 minutes

A sequence question you can name

You can now take a biological string and choose a computation that matches the question. Distance and dynamic programming compare two sequences you already hold. Seeds decide whether a database sequence deserves that comparison. K-mers merge reads into a contig. Frames and a digest connect a DNA string to a peptide a mass spectrometer might see. Those are 5 mechanisms, and a wrong frame or a missed seed is a failure you can name.

A result is not done when a number comes out. You still have to say which string, which frame, and which score produced it. An assembly contig of length 6 that you then translate in frame 0 is a different object from the same bases aligned as DNA against a reference. If the protein alignment scores 8 and a second frame scores 1, you report frame 0 and the score 8, and you name the seed length.

Pick the computation by the question.

Edit distance for two strings

A seed before the full matrix

K-mers when the input is reads

A frame, then a peptide mass

Contig, frame, and a seed that cannot exist

1. Reads ATGCT and GCTA at k = 3 assemble to ATGCTA, length 6.

2. Frame 0 of a different string, ATGAAA, translates to MK. That peptide is not a property of the DNA contig until you translate it.

3. A DNA search seed of length 11 cannot occur inside a contig of length 6, so the seed stage rejects the contig before any score.

Problem 1. You assemble ATGCT and GCTA at k = 3, then search the contig as DNA with seed length 11. What string did assembly spell, and why does the seed stage reject it?

Answer. Assembly spells ATGCTA, length 6, by sharing the 3-mer GCT. A DNA seed has length 11, and 6 is shorter than 11, so the contig contains no seed. The search seed and the assembly k are different word lengths, and this query fails the seed gate.

Problem 2. Frame 0 of ATGAAA is MK. Your toy matrix scores identical letters at 2. What is the score of MK against MK, and why is frame 1 not allowed to use that score?

Answer. M against M scores 2 and K against K scores 2, so the sum is 4. Frame 1 regroups the bases and does not produce MK, so it cannot claim the score 4. You report the frame together with the score.